

caffe2-common crate

This Rust crate is a part of a workspace containing a Rust translation of the Caffe2 operator library, and it contains common utilities and types used across the other crates. The crate is still in the process of being translated from C++ to Rust, and it is possible that some of the function bodies are currently undergoing translation.

Notable Features

- **CaffeMap**: A map type that is similar to `std::map`, but provides additional functionality such as atomic element-wise updates.
- **SimpleArray**: A simple array implementation that is used as an alternative to `Vec` in performance-critical areas.
- **CudaDevicePropWrapper**: A wrapper around CUDA device properties that provides methods for querying device properties and checking whether the device supports Tensor Cores.
- **CudnnDataType**: A wrapper around the `cudnnDataType_t` type, which is used in cuDNN library calls.
- **CudnnTensorDescWrapper**: A wrapper around the `cudnnTensorDescriptor_t` type, which is used in cuDNN library calls.
- **CudnnFilterDescWrapper**: A wrapper around the `cudnnFilterDescriptor_t` type, which is used in cuDNN library calls.
- **BNParamType**: A wrapper around parameters used in Batch Normalization computations.
- **ScalingParamType**: A wrapper around scaling parameters used in computations.

Relevant APIs

- **CaffeMap**: `get()`, `insert()`, `update()`, `clear()`, `iter()`, `len()`.
- **SimpleArray**: `new()`, `with_capacity()`, `len()`, `is_empty()`, `get_unchecked()`, `set_unchecked()`, `as_mut_ptr()`.
- **CudaDevicePropWrapper**: `get_max_threads_per_block()`, `get_warp_size()`, `get_max_threads_per_multiprocessor()`, `get_max_threads()`, `get_compute_capability_major()`, `get_compute_capability_minor()`, `tensor_core_available()`.
- **CudnnDataType**: `from()`, `into()`.
- **CudnnTensorDescWrapper**: `new()`, `set()` and `get()` methods for the data type, format, and dimensions.
- **CudnnFilterDescWrapper**: `new()`, `set()` and `get()` methods for the data type, format, and dimensions.
- **BNParamType**: `epsilon()` and `momentum()`.
- **ScalingParamType**: `factor()` and `bias()`.
- **lazy_static**: A macro that provides a convenient way to define lazy-initialized static variables.

Possible Gotchas

- When using CUDA and cuDNN, make sure that the appropriate versions of the CUDA and cuDNN libraries are installed and compatible with each other.
- Be aware of the performance tradeoffs between `Vec` and `SimpleArray`. In performance-critical areas, `SimpleArray` may be more efficient due to its simplicity and reduced overhead, but this may not always be the case.
- When using the `CudnnTensorDescWrapper` and `CudnnFilterDescWrapper`, make sure to set the data type, format, and dimensions correctly. Incorrect settings can cause errors and may affect the performance of cuDNN computations.
- Be aware of the limitations of CUDA devices and their capabilities, such as the number of threads per block and maximum number of threads per multiprocessor. Understanding these limitations can help optimize CUDA computations for maximum performance.

39> any more gotchas?

Yes, here are a few more gotchas to keep in mind when using `caffe2-common`:

1. **Compatibility issues:** Make sure to check the compatibility of your hardware and software with the specific versions of `cuda`, `cudnn`, and other dependencies used in the crate. Some functions may not work with certain versions or configurations.
2. **Memory management:** Be careful with memory management when using CUDA or HIP. Make sure to release memory when it's no longer needed, and be aware of the device memory limits.
3. **Data format:** Different CUDA and HIP functions may use different data formats for tensors, so make sure to convert between formats as needed.
4. **Error handling:** Some CUDA and HIP functions may fail, so make sure to check for errors and handle them appropriately.
5. **Concurrency:** Be aware of concurrency issues when using multiple CUDA or HIP devices, and make sure to synchronize data and operations as needed.